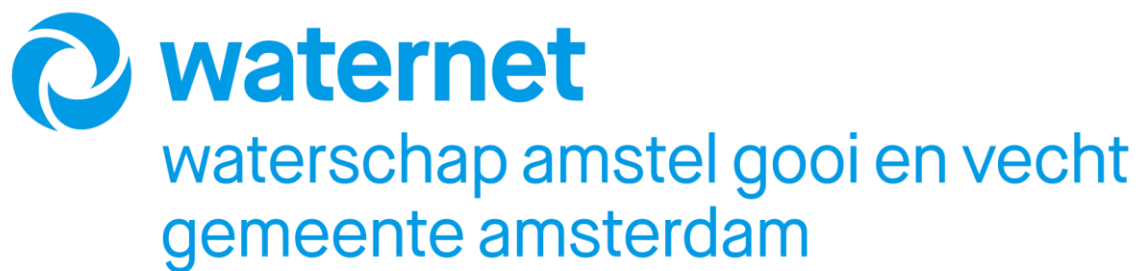


JUNE 16, 2025



DOCUMENTATIE WATERNET

VOORSPELLINGSMODEL DEELTIJDGROEP

CRAIG HAVELOCK, JEROEN HUISKES, WESLEY LANGELAAN, TATJANA VOLKOVA
HOGESCHOOL VAN AMSTERDAM
Project Big Data en Engineering

Inhoudsopgave

Data opschonen en voorbereiding	3
Voorbereiden van de momentopnames:.....	5
Data ophalen:.....	5
Snapshot aanmaken voor elke 3 maanden:	5
Cross join snapshots met leidingen	6
Leeftijd beraken van de leiding	6
Storing binnen drie maanden bekijken	6
Storinggegevens toevoegen	8
Data ophalen uit de database	8
Data samenvoegen tot een dataframe	8
Verschil berekenen tussen snapshot datum en storingsdatum	8
Filteren op storingen tot 6 maanden geleden	9
Gewogen storing berekenen.....	9
Groeperen op pipeid en snapshot.....	9
Samenvoegen	10
Dataframe wegschrijven naar een tabel in de database	10
Weerinformatie toevoegen	11
Date veld omzetten naar datetime.....	11
Filteren op begin en einddatum	11
Nieuwe dataframe.....	11
Opslaan in de database	12
Gemiddelde temperatuur uitrekenen.....	12
Aantal vorstdagen drie weken voor de snapshot	13
Dataframe wegschrijven naar een tabel in de database	13
Aantal lekkages drie maanden voor het momentopname	14
Ophalen snapshots van de database	14
Lekkages ophalen uit de database.....	14
Left join snapshots en lekkages.....	14
Verschil in dagen uitrekenen	15
Samenvoegen	15
Dataframe wegschrijven naar een tabel in de database	15
Features toevoegen	16
Een snapshot gebruiken om te voorspellen	16

Model trainen	17
---------------------	----

Data opschonen en voorbereiding

Voordat de data kon worden gebruikt voor verdere analyse en modelontwikkeling, moest deze eerst worden opgeschoond en geschikt worden gemaakt voor opslag in de database. Een belangrijk onderdeel van deze opschoning betrof de kolommen `inst_date`, `rehab_date` en `rem_date`, die elk jaartallen bevatten. In deze kolommen zijn alle waarden die gelijk zijn aan 0 vervangen door NaN, aangezien deze nullen geen geldige datums representeren. Vervolgens zijn alle drie de kolommen omgezet naar hetzelfde datumformaat (%Y), dat alleen het jaar bevat.

```
# Alle leidingen:
df_pipes = pd.read_csv('Data/pipes_utilb_std.csv')

df_pipes['inst_date'] = df_pipes['inst_date'].replace(0, np.nan)
df_pipes['rehab_date'] = df_pipes['rehab_date'].replace(0, np.nan)
df_pipes['rem_date'] = df_pipes['rem_date'].replace(0, np.nan)

df_pipes['inst_date'] = pd.to_datetime(df_pipes['inst_date'], format='%Y')
df_pipes['rehab_date'] = pd.to_datetime(df_pipes['rehab_date'], format='%Y')
df_pipes['rem_date'] = pd.to_datetime(df_pipes['rem_date'], format='%Y')
df_pipes = df_pipes.drop(['dataset'], axis=1)
df_pipes.to_sql(name='pipes_original', con=engine, if_exists='fail', index=False)
```

In het onderstaande deel van het script worden verschillende datasets ingelezen, opgeschoond en voorbereid voor opslag in de SQL-database. Eerst worden alleen de actieve leidingen uit de oorspronkelijke dataset gefilterd. Vervolgens worden twee externe CSV-bestanden ingelezen, waarbij onnodige tekens uit bepaalde kolommen worden verwijderd en de datatypes worden aangepast voor verdere verwerking. Ook worden irrelevante kolommen verwijderd. Daarna worden de opgeschoonde gegevens in afzonderlijke tabellen in de database geplaatst en vindt een samenvoeging plaats op basis van een gemeenschappelijke sleutel om een gecombineerde dataset te creëren voor verdere analyse.

```
# Alle actieve
df_active_pipes = df_pipes.loc[lambda df: df['active'] == True]

df2 = pd.read_csv('data/breaks_utilb_pipeid_match.csv')
df2['brid'] = df2['brid'].str.replace(',', '', regex=True)
df2['brid'] = pd.to_numeric(df2['brid'])
df2 = df2.drop(['dataset'], axis=1)
df2.to_sql(name='breaks_pipes_match_original', con=engine, if_exists='fail', index=False)

df_with_id = pd.merge(df2, df_pipes, on='pipeid', how='inner')

df3 = pd.read_csv('data/breaks_utilb_csv3.csv')
df3['date'] = pd.to_datetime(df3['date'], format='%m/%d/%y')
df3['brid'] = df3['brid'].str.replace(',', '', regex=True)
df3['brid'] = pd.to_numeric(df3['brid'])
df3.to_sql(name='breaks_original', con=engine, if_exists='fail', index=False)
```

In het onderstaande gedeelte van het script wordt de storingendataset gecombineerd met de leidingeninformatie op basis van een gemeenschappelijk ID. Daarna wordt de datum van de storing geconverteerd naar een geschikt datumformaat. Overbodige kolommen worden verwijderd en leidingen zonder geldige installatiedatum worden gefilterd. Ontbrekende waarden in specifieke kolommen worden vervangen door standaardwaarden zoals 'Geen' of 'onbekend'. Vervolgens worden rijen met ontbrekende of dubbele gegevens verwijderd. Tot slot worden de datumkolommen inst_date, rehab_date en rem_date omgezet naar een uniform jaartalformaat om consistentie in de tijdsgegevens te waarborgen.

```
# Inner join de storing df met pipes DF
df_with_breaks = pd.merge(df_with_id, df3, on='brid', how='inner')

# converteer datum van storing naar datum type:
df_with_breaks['date'] = pd.to_datetime(df_with_breaks['date'], format='%m/%d/%y')

# Kolommen verwijderen die niet belangrijk lijken:
df_with_breaks = df_with_breaks.drop(['brid', 'col_0', 'dataset_x', 'dataset_y', 'loc1_x', 'owner', 'Nummer afhandeling (Afhandeling)', 'loc1_y'], axis=1)

# Leidingen met onbekende installatiedatum verwijderen
df_with_breaks = df_with_breaks[df_with_breaks['inst_date'] != 0]

# Aannames, leeg is geen:
df_with_breaks['cor_protect'] = df_with_breaks['cor_protect'].fillna('Geen')
df_with_breaks['lining'] = df_with_breaks['lining'].fillna('Geen')

df_with_breaks['manufacturer'] = df_with_breaks['manufacturer'].fillna('onbekend')

# Drop velden met missende waarden:
df_with_breaks = df_with_breaks.dropna()

# Duplicates verwijderen:
df_with_breaks = df_with_breaks.drop_duplicates()

# Converteer naar datum:
df_with_breaks['inst_date'] = pd.to_datetime(df_with_breaks['inst_date'], format='%Y')
df_with_breaks['rehab_date'] = pd.to_datetime(df_with_breaks['inst_date'], format='%Y')
df_with_breaks['rem_date'] = pd.to_datetime(df_with_breaks['inst_date'], format='%Y')
```

Na het uitvoeren van deze stappen was de data volledig opgeschoond en in het juiste formaat opgeslagen, zodat we deze direct konden gebruiken voor het ontwikkelen van het voorspellingsmodel.

Vorbereiden van de momentopnames:

Data ophalen:

De eerste stap is het ophalen van de data uit de MySQL database die op OEGE staat. Dit gebeurt via Pandas read_sql. De data wordt opgeslagen in een Pandas dataframe

```
# Connectiestring om met OEGE te verbinden:
engine = connection()

# MySQL queries voor leidingen en lekkages:
queryPipes = """
    SELECT pipeid,dia,mat,len,cor_protect,joints,inst_date ,rehab_date, water_type, roughness, pressure_class, inside_dia, outside_dia
    FROM zlangelw.pipes_original
    """

queryBreakage = """
    SELECT a.pipeid, c.date FROM zlangelw.breaks_pipes_match_original a
    JOIN zlangelw.pipes_original b ON a.pipeid = b.pipeid
    JOIN zlangelw.breaks_original c ON a.brid = c.brid
    WHERE c.isbr = 1
    """

# SQL tabel inlezen in een DF:
df_pipes = pd.read_sql(queryPipes, con=engine)
df_breaks = pd.read_sql(queryBreakage, con=engine)
```

Snapshot aanmaken voor elke 3 maanden:

Vervolgens wordt er een dataframe aangemaakt met de snapshots die we gaan gebruiken. Dit hebben we gedaan met de functie date_range van Pandas met de frequentie van 3 maanden. De start datum is 1 januari 2000 en de einddatum is 1 januari 2021.

```
# Snapshot aanmaken voor elke 3 maanden en omzetten naar DF:
snapshots = pd.date_range(start="2000-01-01", end="2021-01-01", freq="3MS")
snapshots_df = pd.DataFrame({'snapshot': snapshots})
```

Dit geeft het volgende resultaat:

0	2000-01-01 00:00:00
1	2000-04-01 00:00:00
2	2000-07-01 00:00:00
3	2000-10-01 00:00:00
4	2001-01-01 00:00:00
5	2001-04-01 00:00:00
6	2001-07-01 00:00:00
7	2001-10-01 00:00:00
8	2002-01-01 00:00:00
9	2002-04-01 00:00:00
10	2002-07-01 00:00:00
11	2002-10-01 00:00:00
12	2003-01-01 00:00:00
13	2003-04-01 00:00:00
14	2003-07-01 00:00:00
15	2003-10-01 00:00:00
16	2004-01-01 00:00:00
17	2004-04-01 00:00:00
18	2004-07-01 00:00:00

Cross join snapshots met leidingen

Om ervoor te zorgen dat er voor elke leiding snapshots zijn doen we een cross join met leidingen en de snapshots. De cross join zorgt voor $85 \text{ (aantal snapshots)} \times 165429 \text{ (aantal leidingen)}$ records in het nieuwe dataframe `snapshots_all`.

```
# Cross join snapshots met pipes (85 x 165429)
snapshots_all = df_pipes.merge(snapshots_df, how="cross")
```

Het is niet erg dat er snapshots zijn van leidingen die op dat moment nog niet zijn geïnstalleerd. Die worden in de volgende stap verwijderd.

Leeftijd berkenen van de leiding

De leeftijd van een leiding is een belangrijke factor bij het voorspellen of een leiding mogelijk gaat breken. Daarom hebben we voor elke leiding berekend hoe oud deze was op het moment van elke snapshot. Dit doen we door het verschil in dagen te nemen tussen de snapshotdatum en de installatiedatum van de leiding. Het resultaat wordt vervolgens gedeeld door 365.25 om de leeftijd in jaren te verkrijgen.

Als de snapshotdatum eerder is dan de installatiedatum, dan is de berekende leeftijd negatief. Daarom filteren we het dataframe op age gelijk of groter dan 0.

```
# Leeftijd berekenen van de leiding
snapshots_all["age"] = (snapshots_all["snapshot"] - snapshots_all["inst_date"]).dt.days / 365.25

# Alleen positieve getallen voor leeftijd, alleen snapshots wanneer de leiding al 'geboren' is
snapshots_all = snapshots_all[snapshots_all["age"] >= 0]

# Inst_date is niet meer nodig, leeftijd is al berekend
snapshots_all.drop('inst_date', axis=1)
```

Storing binnen drie maanden bekijken

We willen gaan voorspellen of een bepaalde leiding een storing gaat krijgen binnen nu en drie maanden. In ons dataframe wordt een kolom toegevoegd 'storing_binnen_3m'. Wanneer een leiding tussen snapshot en snapshot_end een lekkage heeft gehad, dan wordt 'storing_binnen_3m' op 1 gezet.

Eerst voegen we een kolom 'snapshot_end' toe door bij snapshot 3 maanden op te tellen.

```
# Einde van snapshot toevoegen, einde van de window van 3 maanden
snapshots_all["snapshot_end"] = snapshots_all["snapshot"] + pd.DateOffset(months=3)
```

Daarna doen we een merge tussen snapshots_all (de snapshots met leidingen en berekende leeftijd) en df_breaks (het dataframe met alle lekkages).

We gebruiken een left join, zodat alle rijen uit snapshots_all behouden blijven.

Omdat er meerdere snapshots per leiding zijn (bijvoorbeeld 85 snapshots per leiding), en een leiding ook meerdere lekkages kan hebben (bijvoorbeeld 2), worden de lekkages meerdere keren toegevoegd aan de nieuwe dataframe merged.

Dit komt doordat we joinen op pipeid, en dus elke lekkage aan alle snapshots van die leiding gekoppeld wordt.

```
# Left join snapshots en breuken. Een breuk gevonden, voegt hij deze 85 keer toe (aantal snapshots)
merged = snapshots_all.merge(df_breaks, on="pipeid", how="left")
```

Nu hebben we alle lekkages gekoppeld aan de snapshots. De volgende stap is om alleen de lekkages eruit te filteren die binnen snapshot en snapshot_end vallen. Deze gefilterde dataframe slaan we op als apart dataframe filtered.

```
# Alleen wanneer de breuk datum binnen snapshot en einde snapshot ligt
mask = (merged["date"] >= merged["snapshot"]) & (merged["date"] < merged["snapshot_end"])
filtered = merged[mask]
```

Om te bepalen of de leiding binnen drie maanden van de snapshot een lekkage heeft gehad, groeperen we de gefilterde data op pipeid en snapshot en tellen we het aantal lekkages. Als er een of meer lekkages zijn, geven we dat aan met een nieuwe kolom storing_binnen_3m, die we op 1 zetten. Dit betekent: deze leiding had binnen deze snapshotperiode (van 3 maanden) een lekkage.

Daarna voegen we deze informatie toe aan de oorspronkelijke snapshots_all dataframe met een left join op pipeid en snapshot.

```
# Groepeer per leiding en snapshot en telt het aantal lekkages
has_storing = filtered.groupby(["pipeid", "snapshot"]).size().reset_index(name="count")
has_storing["storing_binnen_3m"] = 1 # Leiding heeft lekkage gehad binnne 3 maanden

# Samenvoegen
snapshots_all = snapshots_all.merge(
    has_storing[["pipeid", "snapshot", "storing_binnen_3m"]],
    on=["pipeid", "snapshot"],
    how="left"
)
```

De leidingen die geen lekkage hebben gehad hebben nu een null waarde voor het kolom. Die vullen we met 0.

```
# Leidingen zonder lekkage op 0 zetten
snapshots_all["storing_binnen_3m"] = snapshots_all["storing_binnen_3m"].fillna(0).astype(int)
```


Storinggegevens toevoegen

Om te voorspellen of er een mogelijke lekkage gaat komen is het nuttig om het aantal meldingen voorafgaand aan het momentopname op te slaan (6 maanden ervoor). Mogelijk gaat een lekkage vooraf aan meldingen over het waterkwaliteit. Meldingen dichtbij de snapshot zijn relevanter dan meldingen verder uit het verleden, vandaar dat een recentere melding een hoger getal krijgt. Ook het aantal meldingen worden opgeteld en opgeslagen.

Data ophalen uit de database

Eerst halen we de storingsgegevens op uit de database en slaan we deze op in een Dataframe `storingen_df`. Deze gegevens gaan we toevoegen aan het dataframe met de snapshots. De snapshots zelf halen we ook op uit de database, die we in “Voorbereiden van de momentopnames” hebben opgeslagen in de SQL database van OEGE.

```
# Connectie met database
engine = connection() # Oege

# TODO Mogelijk meer cleanen
query = """SELECT distinct c.pipeid, a.date, event_class FROM zlangelw.breaks_original a
inner join zlangelw.breaks_pipes_match_original b on a.brid = b.brid
inner join zlangelw.pipes_original c on b.pipeid = c.pipeid
where isbr = 0 and (event_class != 'Andere melding' and event_class != 'Meteropname' and event_class != 'Diversen' and event_class != 'Lekkage')"""

# Snapshots gemaakt in 01 ophalen
query2 = "SELECT * FROM zlangelw.snapshots_part1"

storingen_df = pd.read_sql(query, con=engine)
snapshots_df = pd.read_sql(query2, con=engine)
```

Data samenvoegen tot een dataframe

De tweede stap van dit proces is het samenvoegen van de twee dataframes. Dit hebben we gedaan met een left join op ‘pipeid’. Voor elke leiding zijn er 85 snapshots aanwezig. Wanneer er een storing (geen lekkages) is gevonden wordt de regel 85 keer toegevoegd omdat er 85 keer een match is tussen ‘pipeid’s’. Dit zorgt voor een groot dataframe die we later weer gaan filteren.

```
# Twee DF joinen. Alle snapshots blijven behouden en voegt evt storing toe
merged = snapshots_df.merge(storingen_df, on='pipeid', how='left')
```

Verschil berekenen tussen snapshot datum en storingsdatum

Omdat we graag allen de meldingen van 6 maanden vooraf aan de snapshot willen meenemen moeten we het verschil in tijd berekenen tussen ‘date’ van de melding en ‘snapshot’. Het verschil in dagen wordt tijdelijk opgeslagen in een kolom zodat we daarmee kunnen filteren.

```
# Bereken verschil in dagen tussen snapshot en storing datum
merged['dagen_voor_snapshot'] = (merged['snapshot'] - merged['date']).dt.days
```

Filteren op storingen tot 6 maanden geleden

In deze stap maken we een nieuw dataframe aan met de gefilterde data waarin alleen storingen worden meegenomen die tot 6 maanden voor de snapshot zijn gebeurd.

```
# Filter alleen storingen in de 6 maanden (ongeveer 183 dagen)
filtered = merged[(merged['dagen_voor_snapshot'] > 0) & (merged['dagen_voor_snapshot'] <= 183)]
```

Gewogen storing berekenen

De formule die we hebben gebruikt om dit te berekenen is:

$$1 / (\text{dagen_voor_snapshot} + 1)$$

Deze zorgt ervoor dat een storing die 0 dagen geleden plaatsvond een gewogen score van 1 krijgt. Bij een storing van 10 dagen geleden is de score $1 / (10 + 1) = 0.09$.

De +1 in de noemer voorkomt een deling door nul wanneer de storing op exact dezelfde dag als de snapshot heeft plaatsgevonden.

```
# Gewogen score: gewicht = 1 / (dagen + 1), zo krijgen storingen dichtbij de snapshot meer waarde
filtered['gewogen_score'] = 1 / (filtered['dagen_voor_snapshot'] + 1)
```

Groeperen op pipeid en snapshot

In deze stap groeperen we het DataFrame op pipeid en snapshot, zodat elke combinatie van leiding en snapshot uniek is.

We tellen vervolgens het aantal storingen per groep door het aantal rijen in de kolom dagen_voor_snapshot te tellen (dit had ook met bijvoorbeeld pipeid gekund, omdat elke rij een storing voorstelt).

Daarnaast tellen we de gewogen scores van de storingen bij elkaar op om één totaalscore per snapshot te berekenen.

```
# Groepeer: som per leiding_id + snapshot_date
result = filtered.groupby(['pipeid', 'snapshot']).agg(
    storingen_6m=('dagen_voor_snapshot', 'count'),
    storingen_gewogen=('gewogen_score', 'sum')
).reset_index()
```

Samenvoegen

De laatste stap is een left join tussen snapshots_df en result, het DataFrame met de berekende storingstellingen per snapshot. Als er een match is op pipeid en snapshot, worden de bijbehorende waarden toegevoegd aan de rij.

Voor leidingen zonder match worden de ontbrekende waarden (nulls) vervangen door 0, zodat alle snapshots behouden blijven en storingswaarden altijd numeriek zijn.

```
# Merge terug in snapshots_df
snapshots_df = snapshots_df.merge(result, on=['pipeid', 'snapshot'], how='left')

# Vul lege waarden (geen storingen) met 0
snapshots_df[['storingen_6m', 'storingen_gewogen']] = snapshots_df[['storingen_6m', 'storingen_gewogen']].fillna(0)
```

Dataframe wegschrijven naar een tabel in de database

De snapshots worden opgeslagen in de database

```
# Schrijven naar DB
print(f"[{datetime.now().strftime('%Y-%m-%d %H:%M:%S')}] Writing DF to SQL")

snapshots_df.to_sql("snapshots_part2",
                    con=engine,
                    chunksize=5000,
                    method="multi",
                    if_exists='fail',
                    index=False)
```

Weerinformatie toevoegen

We hebben via de KNMI-website de dataset gedownload met historische weergegevens gemeten op Schiphol. Uit deze dataset hebben we de volgende velden gebruikt:

- TN : Minimale temperatuur
- TX: Maximale temperatuur
- TG: Gemiddelde temperatuur
- RH: Aantal mm regen

Deze velden hebben we toegevoegd aan de snapshots.

Date veld omzetten naar datetime

Omdat we willen filteren op datum hebben we eerst het date veld omgezet naar een datetime veld. Dit hebben we gedaan met de Pandas functie `to_datetime`:

```
# Datum veld naar datetime
df_weer['date'] = pd.to_datetime(df_weer['YYYYMMDD'], format='%Y%m%d')
```

Filteren op begin en einddatum

Omdat we alleen de weerinformatie nodig hebben van 01-12-1999 tot 31-12-2020 hebben we de data gefilterd op die twee datums. Deze gefilterde data hebben we opgeslagen in het dataframe.

```
# Alleen weer data van:
start_date = '1999-12-01'
end_date = '2020-12-31'

# Filter dataframe op start en end date:
df_weer = df_weer[(df_weer['date'] >= start_date) & (df_weer['date'] <= end_date)]
```

Nieuwe dataframe

De volgende stap zetten we alleen de data die we nodig hebben in een nieuw dataframe genaamd `df_selected`. Omdat de kolomnamen niet zoveel zeggen hernoemen we deze naar duidelijkere kolomnamen.

```
# Nieuwe DF van deze velden
selected_columns = ['date', 'TN', 'TX', 'TG', 'RH']
df_selected = df_weer[selected_columns].copy()

# Kolommen hernoemen
df_selected.rename(columns={'TN': 'min_temp'}, inplace=True)
df_selected.rename(columns={'TX': 'max_temp'}, inplace=True)
df_selected.rename(columns={'TG': 'avg_temp'}, inplace=True)
df_selected.rename(columns={'RH': 'rain_amount'}, inplace=True)
```

Opslaan in de database

Om de gegevens nog een keer her te gebruiken slaan we de weerdata ook op in een tabel in de database.

```
# Weer info naar DB:
df_selected.to_sql('weather_info',
                    con=engine,
                    index=False,
                    chunksize=5000,
                    if_exists='fail')
```

Gemiddelde temperatuur uitrekenen

Als eerste voegen we een kolom toe aan het snapshots dataframe die bepaalt in welke range we het gemiddelde moeten uitrekenen. Dit doen we door 21 dagen van de snapshots af te halen met pandas Timedelta functie.

```
# Drie weken voor de snapshot berekenen
df_snapshots['start_date'] = df_snapshots['snapshot'] - pd.Timedelta(days=21)
```

We maken gebruik van de library DuckDB om de gemiddelde temperatuur drie weken voor de snapshot uit te rekenen. Met deze library kan je SQL queries uitvoeren op Pandas dataframe. We rekenen met de avg functie in duckdb het gemiddelde uit van de temperatuur drie weken voor de snapshot (Datum weer is groter of gelijk dan start_date en Datum weer is kleiner dan Snapshot datum).

```
# Query om gemiddelde temp uit te rekenen
result = duckdb.query("""
    SELECT
        s.snapshot,
        AVG(w.avg_temp) AS avg_temp_3w
    FROM df_snapshots s
    JOIN df_selected w
    ON w.date >= s.start_date AND w.date < s.snapshot
    GROUP BY s.snapshot
""").to_df()
```

Deze rijen voegen we met een left join weer toe aan de snapshots.

```
# Merge met df_snapshots
df_snapshots = df_snapshots.merge(result, on='snapshot', how='left')
```

Aantal vorstdagen drie weken voor de snapshot

Hetzelfde passen we toe met het aantal vorstdagen. Het verschil is dat we geen avg functie gebruiken maar een count omdat we dagen moeten tellen. De velden met een null waarde vullen we na afloop met 0

```
# Aantal vorstdagen
result_frost = duckdb.query("""
    SELECT
        s.snapshot,
        COUNT(DISTINCT w.date) AS frost_3w
    FROM df_snapshots s
    JOIN df_selected w
        ON w.date >= s.start_date AND w.date < s.snapshot
    WHERE w.min_temp < 0.0
    GROUP BY s.snapshot
""").to_df()

# Merge
df_snapshots = df_snapshots.merge(result_frost, on='snapshot', how='left')
# Geen vorst, 0 invullen
df_snapshots['frost_3w'].fillna(0, inplace=True)
```

Dataframe wegschrijven naar een tabel in de database

De snapshots worden opgeslagen in de database

```
# Wegschrijven naar database
df_snapshots.to_sql('snapshots_part3',
                    con=engine,
                    index=False,
                    chunksize=5000,
                    if_exists='fail'
                    )
```

Aantal lekkages drie maanden voor het momentopname

De kans op een nieuwe lekkage is misschien groter als er al lekkages zijn geweest drie maanden voor het momentopname. In deze stap tellen we het aantal lekkages op en voegen we dit toe aan een kolom in het dataframe om deze vervolgens weg te schrijven naar de database.

Ophalen snapshots van de database

De eerste stap is het ophalen van de snapshots uit de database die we in de stap hiervoor hebben weggeschreven. Aan dit dataframe wordt later een extra kolom toegevoegd.

```
# Snapshots 03 ophalen:
df_snapshots = pd.read_sql("SELECT * FROM zlangelw.snapshots_part3",
                             con=engine
                             )
```

Lekkages ophalen uit de database

Vervolgens halen we de unieke lekkages op uit de database. De lekkages zijn uniek op pipeid, datum en event_class (Lekkage). Hierdoor worden twee lekkage meldingen van dezelfde dag gezien als dezelfde lekkage. Deze lekkages worden in een eigen dataframe opgeslagen en hierna gemerged.

```
# Query voor de lekkages
query = """SELECT distinct c.pipeid, a.date, event_class
FROM zlangelw.breaks_original a
inner join zlangelw.breaks_pipes_match_original b on a.brid = b.brid
inner join zlangelw.pipes_original c on b.pipeid = c.pipeid
where isbr = 1
"""

# Lekkages ophalen
df_breakages = pd.read_sql(query,
                             con=engine
                             )
```

Left join snapshots en lekkages

In deze stap voegen we de twee dataframes samen met een left join. Alles wat er in de snapshots staat blijft, ook al zijn er geen lekkages, en worden de lekkages toegevoegd als er een match is tussen pipeid en datum.

```
# Voeg alle combinaties samen op pipeid
df_merged = df_snapshots[['pipeid', 'snapshot']].merge(
    df_breakages[['pipeid', 'date']],
    on='pipeid',
    how='left'
)
```

Vershil in dagen uitrekenen

Hier bereken we het verschil in dagen tussen de snapshot date en de datum van de lekkage. Uiteindelijk willen we alle rijen behouden (en tellen) wanneer het verschil in dagen gelijk of minder is dan 90 dagen. Hierbij hebben we dat het aantal lekkages drie maanden voor de snapshot berekend. Dit resultaat slaan we op in een dataframe `df_filtered`.

```
# Bereken het verschil in dagen
df_merged['days_diff'] = (df_merged['snapshot'] - df_merged['date']).dt.days

mask = (df_merged['days_diff'] > 0) & (df_merged['days_diff'] <= 90)
df_filtered = df_merged[mask]
```

Samenvoegen

We groeperen de gefilterde dataframe op pipeid en snapshot. Daarna tellen we hoe vaak deze combinatie voorkomt. Het resultaat hiervan slaan we op in het kolom `breakages_last_3m`. Daarna doen we een left join tussen snapshots en `breakage_counts` om het aantal lekkages toe te voegen. De kolommen met een null waarde hebben geen lekkage gehad, deze vullen we met 0

```
# Groepeer en tel per snapshot
breakage_counts = (
    df_filtered.groupby(['pipeid', 'snapshot'])
    .size()
    .reset_index(name='breakages_last_3m')
)

# Voeg terug aan originele df_snapshots
df_snapshots = df_snapshots.merge(breakage_counts, on=['pipeid', 'snapshot'], how='left')

# Vul NaN (geen lekkages) met 0
df_snapshots['breakages_last_3m'] = df_snapshots['breakages_last_3m'].fillna(0).astype(int)
```

Dataframe wegschrijven naar een tabel in de database

De snapshots worden opgeslagen in de database

```
# Naar DB schrijven:
df_snapshots.to_sql("snapshots_part4", con=engine, chunksize=5000, method="multi", if_exists='fail', index=False )
```


Features toevoegen

In deze stap voegen we meer feature toe aan de snapshots tabel.

- Tijd sinds 'rehab' berekenen.
- Storing impact score toevoegen. Aantal meldigen vermenigvuldigd met de leeftijd, oudere leidingen met veel storingen hebben een hoger risico.
- Hetzelfde voor de gewogen score. Een oude leiding met een hoog gewogen storingen (meer recentere meldingen) hebben een hoger risico.
- Corrosie risico toevoegen wanneer een leiding geen of praktisch geen bescherming heeft
- Sommige materialen hebben een hoger risico. We hebben het percentage van het aantal leidingen (per materiaal) die een storing hebben gehad berekend.
- Vervolgens ook het materiaalrisico vermenigvuldigd met de leeftijd om de materiaal_leeftijds_risico te berekenen.

Een snapshot gebruiken om te voorspellen

De laatste stap voor het voorbereiden is het laatste snapshot apart te houden om te voorspellen. We hebben data tot aan oktober 2020. We gebruiken deze laatste snapshot om te voorspellen. De data van voor oktober 2020 gebruiken we om te trainen en testen.

```
# Laatste snapshot om te predicten als model gemaakt is:
datumgrens = pd.Timestamp("2020-10-01")

df_voor_2020_10_01 = df_snapshots[df_snapshots["snapshot"] < datumgrens].copy()
df_na_2020_10_01 = df_snapshots[df_snapshots["snapshot"] >= datumgrens].copy()

df_na_2020_10_01.to_sql("snapshots_predict", con=engine, chunksize=5000, method="multi", if_exists='fail', index=False )
df_voor_2020_10_01.to_sql("snapshots_trainetest", con=engine, chunksize=5000, method="multi", if_exists='fail', index=False )
```

Model trainen

We moeten eerst de data uit de database ophalen en deze in een Dataframe opslaan zodat we deze kunnen gebruiken om te trainen en testen

```
## Data in DF laden
engine = connection() # Oege

query = """
SELECT dia, mat, len, cor_protect, water_type,
FROM zlangelw.snapshots_traintest
"""

df = pd.read_sql(query, con=engine)
```

Er zijn een aantal leidingen waar materiaal en diameter onbekend is. Deze moeten we verwijderen omdat ons model deze waardes nodig heeft.

```
## Verwijder rijen met Null waardes:
df = df.dropna(subset=['dia'])
df = df.dropna(subset=['mat'])
```

Om ervoor te zorgen dat we niet teveel rijen verliezen door onvolledige data hebben we het kolom cor_protect gevuld met de waarde 'Onbekend' wanneer dit kolom een null waarde bevat.

```
## Corrosie protectie op onbekend zetten, zijn er teveel om te verwijderen
df['cor_protect'] = df['cor_protect'].fillna('Onbekend')
```

XGBoost kan alleen omgaan met numerieke waardes, waardes met categorieën moeten we markeren als een categorie veld.

```
# Velden markeren als categorie, XGB kan alleen omgaan met getallen:
categorical_cols = ['mat', 'cor_protect']
for col in categorical_cols:
    X[col] = X[col].astype('category')
```

De volgende stap is het splitsen van de data in een trainingsset en een testset. Dit doen we op basis van de snapshotdatum, omdat we het model geen informatie uit de toekomst willen laten zien. Zo voorkomen we dat er sprake is van datalek (geen waterleidinglek 😊). Alle data van vóór 2017 gebruiken we om het model te trainen, en de data uit 2018 gebruiken we om het model te testen.

De jaren 2019 en 2020 hebben we buiten beschouwing gelaten, omdat er in de dataset

zeer weinig lekkages uit die jaren zijn opgenomen. We vermoeden dat de data over deze jaren niet compleet is.

```
# Splitten op target en cutoff_date, we willen hem niet de toekomst laten zien
target = 'storing_binnen_3m'
cutoff_date = pd.to_datetime('2017-01-01')

train_df = X[(X['snapshot'] <= cutoff_date)]
test_df = X[X['snapshot'] > cutoff_date]
test_df = test_df[~test_df['snapshot'].dt.year.isin([2019, 2020])] # 2019 en 2020 zijn onvolledig

# Features en target splitsen
X_train = train_df.drop(columns=[target]) # targetweghalen
y_train = train_df[target] # de target

X_test = test_df.drop(columns=[target])
y_test = test_df[target]

# Snapshot is nu niet nodig, droppen voor fitten
X_test = X_test.drop(columns='snapshot')
X_train = X_train.drop(columns='snapshot')
```

Vervolgens trainen we het model om de train data

```
# XGboost model
model = xgb.XGBClassifier(
    max_depth=4,
    min_child_weight=5,
    gamma=0.2,
    scale_pos_weight=50, # Vanwege inbalans in pos / neg
    eval_metric='aucpr',
    use_label_encoder=False,
    enable_categorical=True,
    tree_method='hist'
)

# Trainen
model.fit(X_train, y_train)
```